

Compte-rendu TP7

Spécification et Modélisation de Programme

Lors de ce TP, j'ai pu mieux appréhender la manipulation de classes, travailler sur la surcharge d'opérateurs et mettre en œuvre le polymorphisme.

1. Création d'une classe *Point*

La première étape consiste à créer une classe représentant un point du plan.

Un point est défini par deux coordonnées réelles :

- Une abscisse x ;
- Une ordonnée y .

Cette classe sert de base pour l'ensemble du TP, étant donné qu'elle sera utilisée plus tard pour définir le centre des formes géométriques.

Les coordonnées « x » et « y » sont déclarées *private*.

Ce choix permet de garantir l'encapsulation des données. Les attributs internes ne peuvent pas être modifiés directement depuis l'extérieur de la classe.

L'accès à ces valeurs se fait uniquement par l'intermédiaire :

- D'accesseurs : *getx()* et *gety()*,
- De mutateurs *setx()* et *sety()*.

Cette approche permet de mieux contrôler les modifications et d'éviter des incohérences dans l'état des objets.

Plusieurs constructeurs ont été écrits pour permettre de créer un point de différentes manières :

- Un constructeur sans paramètre, qui initialise le point à l'origine du repère (0,0) ;
- Un constructeur à deux paramètres, permettant de définir explicitement les coordonnées ;
- Un constructeur de copie, prenant en paramètre une référence constante vers un autre objet *Point*.

L'utilisation d'une référence constante (*Point const&*) permet d'éviter les copies inutiles tout en garantissant que l'objet source ne sera pas modifié.

Deux méthodes *translater()* ont été définies :

- L'une prenant deux réels en paramètre ;
- L'autre prenant un objet *Point*.

Ces méthodes permettent de déplacer un point dans le plan, soit par un déplacement vectoriel, soit par l'ajout des coordonnées d'un autre point.

Des jeux d'essais ont permis de vérifier :

- La bonne initialisation des points ;
- Le fonctionnement correct des translations ;
- L'accès et la modification des coordonnées via les accesseurs et mutateurs.

```
olwen@MacBook-Pro-de-Toukan TP7_SMP % ./machin.out
3
2
Coordonnées AVANT translation :
3x et 2y.
2
1
Coordonnées APRES translation :
5x et 3y.
```

Diagramme de classe de la classe « *Point* » :

Point
- float x - float y
// Constructeurs - Point() - Point(float x, float y) - Point(float const&t)
// Accesseurs - float getx()const - float gety()const
// Mutateurs - void sextx(float x) - void sety(float y)
// Méthodes - translater(float x, float y) - translater(Point&) - void print()

2. Surcharge d'opérateurs

L'opérateur << a été surchargé afin de permettre l'affichage direct d'un point à l'aide de cout. Cet opérateur est défini en dehors de la classe Point, car son opérande gauche (*ostream*) n'est pas un objet *Point*.

Il prend en paramètre une référence constante vers un point afin d'éviter toute copie inutile et de garantir l'absence de modification.

Cette surcharge améliore considérablement la lisibilité du code et facilite les tests.

L'opérateur += a été surchargé comme méthode membre de la classe Point. Il permet d'ajouter les coordonnées d'un point à un autre, ce qui correspond à une translation.

Cette surcharge simplifie l'écriture du code et rend les opérations plus naturelles d'un point de vue mathématique.

Les jeux d'essais montrent que :

- L'affichage avec « *cout* » est correct ;
- L'opérateur += modifie bien les coordonnées du point appelant.

```
olwen@MacBook-Pro-de-Toukan TP7_SMP % ./machin.out
5
3
Coordonnées AVANT translation :
5x et 3y.
7
1
Coordonnées APRES translation :
12x et 4y.
```

3. Formes géométriques abstraites

La classe *Forme* représente une forme géométrique abstraite centrée sur un point.

Elle ne correspond pas à une forme concrète, mais définit une « interface commune » pour toutes les formes géométriques.

La classe *Forme* contient un attribut *Point* *o* représentant le centre de la forme.

Cet attribut est déclaré « *private* » afin d'être accessible par les classes dérivées, tout en restant inaccessible depuis l'extérieur.

Deux méthodes abstraites sont déclarées :

- *perimetre()* ;
- *surface()*.

Ces méthodes sont déclarées « virtuelles pures », ce qui rend la classe *Forme* abstraite.

Toute classe dérivée doit obligatoirement fournir une implémentation de ces méthodes.

Une méthode virtuelle « *afficher()* » est déclarée dans *Forme*.

Elle est utilisée par la surcharge de l'opérateur <<, ce qui permet un affichage polymorphe des formes.

Ainsi, une référence ou un pointeur sur *Forme* peut désigner n'importe quelle forme concrète, et l'affichage sera automatiquement adapté au type réel de l'objet.

4. Formes géométriques abstraites

La classe « *Rectangle* » hérite de « *Forme* ».

Elle est définie par :

- Un centre (hérité de *Forme* également) ;
- Une largeur ;
- Une hauteur.

Les côtés du rectangle sont alignés avec les axes horizontaux et verticaux.

La classe implémente :

- Le calcul du périmètre ;
- Le calcul de la surface ;
- L'affichage de ses attributs.

La classe « *Cercle* » hérite directement de « *Forme* ».

Elle est définie par un rayon et implémente les formules mathématiques classiques pour le calcul du périmètre et de la surface.

Les jeux d'essais réalisés permettent de vérifier :

- L'instanciation correcte des différentes formes ;
- La validité des calculs de périmètre et de surface ;
- Le fonctionnement de la translation ;

- L'affichage polymorphe via l'opérateur <<.

```
olwen@MacBook-Pro-de-Toukan TP7_SMP % ./machin.out
Ecrivez la largeur et la hauteur du rectangle :
4
2
Coordonnées d'origine du rectangle : 0x et 0y.
Perimetre = 12
Surface   = 8
Ecrivez la taille du rayon du cercle :
5
Coordonnées d'origine du cercle : 0x et 0y.
Perimetre = 31.4159
Surface   = 78.5398
```